Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo, Norway[1]

February 5, 2025

Plans for the week of February 3-7, 2025

1. Quantum circuits and operations (one-, two- and three-qubit gates
2. Hands-on exercises, see end of these slides
3. Discussion of the first exercise of project 1
4. Video of lecture

Running on the IBM machines

For the projects and exercises, using for example Qiskit, we can run (as of now) on IBM's 127 qubit machine (Osaka). The Bell-state simulations below here perform this. To get started with the IBM machines, you need an account. Here's the recipe you need to follow:

1. Go to `quantum.ibm.com` and create an account and follow the instructions.
2. After you have set up your account, go to the account icon and find the token. You need this to identify your self when running jobs. See the code example below.
3. Clicking on the **jobs** link you can see which machines are availbale and where your job is, whether it is a queue or whether it is pending.

The example at the end of this week's slides shows you how can run on IBM-Osaka, the 127 qubit machine which is presently available. Make sure you copy in your own token, as that is your unique identifier.

Explicit instructions

```
# Install qiskit and various packages
pip install -q qiskit
pip install -q qiskit-aer
pip install -q qiskit-ibmq-provider
```

Setting up your IBM run

```
# Provide your IBM Quantum Experience API token here
# Paste in this from your IBM account
api_token = 'your_api_token'
# Load your IBM Quantum account
IBMQ.save_account(api_token, overwrite=True)  # This stores your crede
IBMQ.load_account()
# Get the least busy backend
provider = IBMQ.get_provider(hub='ibm-q', group ='open', project='main
# Run on the 127-qubit machine IBM-Osaka
backend = provider.get_backend('ibm_osaka')
# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)
```

Quantum gates, circuits and simple algorithms

Quantum gates are physical actions that are applied to the physical system representing the qubits. Mathematically, they are complex-valued, unitary matrices which act on the complex-values normalized vectors that represent qubits. As the quantum analog of classical logic gates (such as AND and OR), there is a corresponding quantum gate for every classical gate; however, there are quantum gates that have no classical counter-part. They act on a set of qubits and, changing their state. That is, if U is a quantum gate and $|q\rangle$ is a qubit, then acting the gate U on the qubit $|q\rangle$ transforms the qubit as follows:

$$|q\rangle \xrightarrow{U} U|q\rangle. \tag{1}$$

Quantum circuits

Quantum circuits are diagrammatic representations of quantum algorithms. The horizontal dimension corresponds to time; moving left to right corresponds to forward motion in time. They consist of a set of qubits $|q_n\rangle$ which are stacked vertically on the left-hand side of the diagram. Lines, called quantum wires, extend horizontally to the right from each qubit, representing its state moving forward in time. Additionally, they contain a set of quantum gates that are applied to the quantum wires. Gates are applied chronologically, left to right.

Single-Qubit Gates

A single-qubit gate is a physical action that is applied to one qubit. It can be represented by a matrix U from the group SU(2). Any single-qubit gate can be parameterized by three angles: θ, ϕ, and λ as follows

$$U(\theta, \phi, \lambda) = \begin{bmatrix} \cos\frac{\theta}{2} & -e^{i\lambda}\sin\frac{\theta}{2} \\ e^{i\phi}\sin\frac{\theta}{2} & e^{i(\phi+\lambda)}\cos\frac{\theta}{2} \end{bmatrix}.$$

Widely used gates

There are several widely used quantum gates. Perhaps the most famous are the Pauli gates correspond to the Pauli matrices

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix},$$

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Algebra basis

These gates form a basis for the algebra $\mathfrak{su}(2)$. Exponentiating them will thus give us a basis for SU(2), the group within which all single-qubit gates live.

Exponentiated Pauli gates

These exponentiated Pauli gates are called rotation gates $R_\sigma(\theta)$ because they rotate the quantum state around the axis $\sigma = X, Y, Z$ of the Bloch sphere by an angle θ. They are defined as

$$R_X(\theta) = e^{-i\frac{\theta}{2}X} = \begin{bmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix},$$

$$R_Y(\theta) = e^{-i\frac{\theta}{2}Y} = \begin{bmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix},$$

$$R_Z(\theta) = e^{-i\frac{\theta}{2}Z} = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}.$$

Basis for $\mathrm{SU}(2)$

Because they form a basis for $\mathrm{SU}(2)$, any single-qubit gate can be decomposed into three rotation gates. Indeed

$$R_z(\phi)R_y(\theta)R_z(\lambda) = \begin{bmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{bmatrix} \begin{bmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{bmatrix} \begin{bmatrix} e^{-i\lambda/2} & 0 \\ 0 & e^{i\lambda/2} \end{bmatrix}$$

which we can rewite as

$$e^{-i(\phi+\lambda)/2} \begin{bmatrix} \cos\frac{\theta}{2} & -e^{i\lambda}\sin\frac{\theta}{2} \\ e^{i\phi}\sin\frac{\theta}{2} & e^{i(\phi+\lambda)}\cos\frac{\theta}{2} \end{bmatrix},$$

which is, up to a global phase, equal to the expression for an arbitrary single-qubit gate.

Two-Qubit Gates

A two-qubit gate is a physical action that is applied to two qubits. It can be represented by a matrix U from the group SU(4). One important type of two-qubit gates are controlled gates, which work as follows: Suppose U is a single-qubit gate. A controlled-U gate (CU) acts on two qubits: a control qubit $|x\rangle$ and a target qubit $|y\rangle$. The controlled-U gate applies the identity I or the single-qubit gate U to the target qubit if the control gate is in the zero state $|0\rangle$ or the one state $|1\rangle$, respectively.

Control qubit

The control qubit is not acted upon. This can be represented as follows if

$$CU|xy\rangle = |xy\rangle \text{ if } |x\rangle = |0\rangle.$$

In matrix form

It is easier to see in a matrix form. It can be written in matrix form by writing it as a superposition of the two possible cases, each written as a simple tensor product

$$CU = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & u_{00} & u_{01} \\ 0 & 0 & u_{10} & u_{11} \end{bmatrix}.$$

CNOT gate

One of the most fundamental controlled gates is the CNOT gate. It is defined as the controlled-X gate CX. It can be written in matrix form as follows:

$$\mathrm{CNOT} = \mathrm{CX} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

CX gate

It changes, when operating on a two-qubit state where the first qubit is the control qubit and the second qubit is the target qubit, the states (check this)

$$\mathrm{CX}|00\rangle = |00\rangle,$$

$$\mathrm{CX}|10\rangle = |11\rangle,$$

$$\mathrm{CX}|01\rangle = |01\rangle,$$

$$\mathrm{CX}|11\rangle = |10\rangle,$$

which you can easily see by simply multiplying the above matrix with any of the above states.

Swap gate

A widely used two-qubit gate that goes beyond the simple controlled function is the SWAP gate. It swaps the states of the two qubits it acts upon

$$\mathrm{SWAP}|xy\rangle = |yx\rangle.$$

and has the following matrix form

$$\mathrm{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Introduction to Qiskit

This part is best seen using the jupyter-notebook.
For the installation

```
pip install qiskit
pip install qiskit-ibm-runtime
```

Documentation can be found at
https://docs.quantum.ibm.com/api/qiskit

Simple code

```
#!/usr/bin/env python
# coding: utf-8
import numpy as np
import qiskit as qk
from scipy.optimize import minimize


# # Initialize registers and circuit

n_qubits = 1 #Number of qubits
n_cbits = 1 #Number of classical bits (the number of qubits you want t
qreg = qk.QuantumRegister(n_qubits) #Create a quantum register
creg = qk.ClassicalRegister(n_cbits) #Create a classical register
circuit = qk.QuantumCircuit(qreg,creg) #Create your quantum circuit


circuit.draw() #Draw circuit. It is empty
```

Thereafter we perform operations on qubit

```
circuit.x(qreg[0]) #Applies a Pauli X gate to the first qubit in the q
circuit.draw()
```

and select a qubit to measure and encode the results to a classical bit

```
#Measure the first qubit in the quantum register
#and encode the results to the first qubit in the classical register
```

Exercises on Bell states

```
import qiskit as qk
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
import matplotlib.pyplot as plt
```

Setting up a circuit

```
qreg_q = QuantumRegister(2) # 2 qubits
creg_c = ClassicalRegister(2) # 2 classical bits

qc = QuantumCircuit(qreg_q, creg_c) # alternatively, circuit = Quantum

qc.h(qreg_q[0]) # apply the hadamard gate to the first qubit (to the r

# apply the CNOT gate with the first qubit being the control qubit and
qc.cx(qreg_q[0], qreg_q[1])


# measure the qubits, specify which qubit you want to measure
qc.measure(qreg_q[0], creg_c[0])
qc.measure(qreg_q[1], creg_c[1])
```

Next we will use an ideal(noiseless) simulator (AerSimulator) to run our circuit. We will use the qasm_simulator backend. This simulator returns a result object which contains the counts, or

Testing the Bell states

```
from qiskit import QuantumCircuit, transpile, assemble, IBMQ
from qiskit.visualization import circuit_drawer
import qiskit_aer
import matplotlib.pyplot as plt

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second
bell_circuit.cx(0, 1)

# Add measurements to the circuit
bell_circuit.measure([0, 1], [0, 1])

# Visualize the circuit
print("Quantum Circuit:")
print(bell_circuit)
```

Running the calculations

```
# Number of shots
num_shots = 10000

# Simulate the circuit using the Aer simulator
simulator = qiskit_aer.Aer.get_backend('qasm_simulator')

# Transpile the circuit for the simulator
transpiled_circuit = transpile(bell_circuit, simulator)

# Execute the transpiled circuit on the simulator with the specified n
result = simulator.run(transpiled_circuit, shots=num_shots).result()

# Get measurement counts
counts = result.get_counts(bell_circuit)

# Get and print the measurement results
counts = result.get_counts(bell_circuit)
print("\nMeasurement Results:")
print(counts)
```

Then plotting the histogram over counts

```
# Plot the histogram using Matplotlib
plt.bar(counts.keys(), counts.values())
plt.title('Bell State Measurement Results')
plt.ylabel('Counts')
plt.show()
```

Getting serious, Quantum Simulator with Hardware noise model

```
from qiskit import QuantumCircuit, transpile, assemble, Aer, IBMQ
from qiskit.visualization import plot_histogram
from qiskit.providers.aer.noise import NoiseModel

# you will not be able to run unless you provide your token here
api_token = 'your_api_token'
# Load your IBM Quantum account
IBMQ.save_account(api_token, overwrite=True)  # This stores your crede
IBMQ.load_account()
# Get the least busy backend
provider = IBMQ.get_provider(hub='ibm-q', group ='open', project='main
#backend = provider.get_backend('your_preferred_backend')
backend = provider.get_backend('ibm_osaka')

# Get the noise model for the selected backend
noise_model = NoiseModel.from_backend(backend)

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second
bell_circuit.cx(0, 1)
```

Then run and visualize it

```
# Number of shots
num_shots = 10000

# Simulate the circuit using the Aer simulator with the noise model
simulator = Aer.get_backend('qasm_simulator')
noisy_transpiled_circuit = transpile(bell_circuit, simulator, optimiza
noisy_circuit = assemble(noisy_transpiled_circuit, shots=num_shots, no

# Execute the noisy circuit on the simulator
result = simulator.run(noisy_circuit).result()

# Get and print the measurement results
counts = result.get_counts(bell_circuit)
print("\nMeasurement Results:")
print(counts)
# Plot the histogram using Matplotlib
plot_histogram(counts, title='Bell State Measurement Results')
```

And without the noise model

```
# Provide your IBM Quantum Experience API token here
api_token = 'your_api_token'
# Load your IBM Quantum account
IBMQ.save_account(api_token, overwrite=True)  # This stores your crede
IBMQ.load_account()

# Get the least busy backend
#provider = IBMQ.get_provider(hub='your_hub', group='your_group', proj
provider = IBMQ.get_provider(hub='ibm-q', group ='open', project='main
#backend = provider.get_backend('your_preferred_backend')
backend = provider.get_backend('ibm_osaka')

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second
bell_circuit.cx(0, 1)

# Add measurements to the circuit
bell_circuit.measure([0, 1], [0, 1])

# Transpile the circuit for the chosen backend
transpiled_circuit = transpile(bell_circuit, backend)

# Execute the transpiled circuit on the IBM Quantum computer
```